MeshRT: Compile-Time Governed Wafer-Scale Runtime for Low-Latency High-Throughput Inference

Congjie He[1] Le Xu[1] Zhan Lu[1] Yeqi Huang[1] Haocheng Xiao[1] Cheng Deng[1] Lingxiao Ma[2]
Ziming Miao[2] Fan Yang[2] Luo Mai[1]
[1]*University of Edinburgh* [2]*Microsoft Research*

Abstract

Wafer-scale accelerators promise ultra-low-latency AI inference, but current system stacks still carry an unsustainably high cost premium. The reason is that many current and emerging inference techniques, such as batching and MoE, introduce runtime dynamism that existing wafer-scale systems cannot support efficiently. As a result, they must choose between two unsatisfactory options: accept suboptimal latency, or preserve low latency by restricting dynamic techniques, thereby limiting the models they can support and reducing aggregate throughput. This tradeoff significantly increases the cost of low-latency serving.

We present MeshRT, the first wafer-scale system to achieve both ultra-low latency and high throughput for LLM inference. MeshRT achieves this via a novel system architecture, which we call *compile-time governance of runtime dynamism*. This architecture features new system abstractions and mechanisms that enable MeshRT to compile dynamic events arising from LLM inference directly into per-core schedules, enabling asynchronous and decentralized execution with low runtime overhead. Implemented on a commodity wafer-scale accelerator, MeshRT preserves ultra-low latency while improving throughput by up to 56× over the state-of-the-art system for wafer-scale accelerators. Against measured single-instance SGLang results on H200, MeshRT achieves 4.5–7.0× lower latency. Under ideal linear scaling of those GPU measurements to an area-normalized H200 cluster, MeshRT achieves 1.6–12.1× higher decode energy efficiency in the low-latency regime.

CCS Concepts: • **Software and its engineering** → **Runtime environments**; • **Computer systems organization** → *Multicore architectures.*

Keywords: Wafer-scale Accelerators, LLM Inference, Compile-time governance, Runtime dynamism



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '26, Prague, Czech Republic

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2585-2/2026/09
https://doi.org/10.1145/3830418.3843856

ACM Reference Format:
Congjie He[1] Le Xu[1] Zhan Lu[1] Yeqi Huang[1] Haocheng Xiao[1] Cheng Deng[1] Lingxiao Ma[2] Ziming Miao[2] Fan Yang[2] Luo Mai[1], [1]*University of Edinburgh* [2]*Microsoft Research* . 2026. MeshRT: Compile-Time Governed Wafer-Scale Runtime for Low-Latency High-Throughput Inference. In *ACM SIGOPS 32nd Symposium on Operating Systems Principles (SOSP '26), September 29–October 02, 2026, Prague, Czech Republic.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3830418.3843856

1 Introduction

Low-latency LLM inference is becoming a core requirement where users and providers are willing to pay a premium for faster responses (§2.1). This drives a growing interest in inference infrastructure that can deliver substantially lower latency than conventional GPU clusters even with small batches.

Wafer-scale accelerators [10] are especially promising for this setting. With massive on-chip SRAM and dense on-chip interconnect, they can reduce the off-chip overhead that dominates inference latency on conventional systems. Yet low-latency serving remains expensive at scale. In practice, providers often operate conservatively: they avoid aggressive batching, limit concurrency, and restrict dynamic inference features that could incur significant overhead. This preserves low latency, but it also leaves much of the hardware underutilized and makes low-latency serving far more expensive than throughput-oriented serving [22].

This cost premium reflects a deeper systems limitation: modern inference workloads increasingly introduce *runtime dynamism*, aspects of execution that the precompiled schedule cannot determine in advance and that must be resolved as inference proceeds. Common examples include changes in active request state during continuous batching or micro-batch pipelining, as well as token-dependent decisions such as Mixture-of-Experts (MoE) routing [16, 17, 25], sparse attention [13, 14, 67, 68], and speculative decoding [30, 34]. On GPU systems, such runtime decisions are typically handled through host-managed runtime state [28, 71]. On wafer-scale accelerators, however, sending them off chip or resolving them through naïve broad synchronization hurts both latency and throughput. Existing wafer-scale systems [22] struggle to provide support for runtime dynamism. This forces users into a no-win trade-off: either keep execution sufficiently static to preserve latency but limit system

throughput, which leads to high serving cost, or support runtime dynamism at a high coordination overhead, hurting serving latency.

We present MeshRT, a system for wafer-scale inference that addresses this challenge through *compile-time governance of runtime dynamism*: critical runtime decisions are governed by compile-time-generated on-chip schedules, rather than by falling back to host control or chip-wide synchronization. Here, "on-chip" refers to the compiled control state installed on the cores; MeshRT's governance is unchanged whether model parameters and KV caches use SRAM alone or an SRAM–DRAM hierarchy. Our approach rests on a key observation: LLM inference uses a closed set of kernel types. Thus, although the specific values of runtime decisions are unknown until execution, the *structure* of dynamism: where it arises, what type it is, and which cores are affected, is fully determinable at compile time. MeshRT exploits this by compiling dynamism handling directly into per-core schedules in a single synthesis pass, and then making these schedules executable under tight per-core resource budgets through safe on-chip reconfiguration. As a result, execution preserves the low-latency benefits of wafer-scale accelerators while supporting runtime dynamism and substantially improving throughput.

MeshRT makes two contributions. First, it introduces *dynamism aware per-core schedule synthesis*: given an LLM computation graph and spatial mapping on a wafer-scale accelerator, the compiler determines, from each kernel's function alone, which cores will receive or produce runtime dynamism, and weaves the corresponding control message handling logic directly into the per-core schedule alongside ordinary data processing. This turns runtime dynamism from an external runtime responsibility into a compile-time scheduling object, providing on-chip runtime control for continuous batching, MoE routing, and micro-batch pipelining. Second, because the synthesized schedules may exceed the limited hardware resources, such as routing resources and queue buffers of individual cores, MeshRT introduces *safe incremental materialization*: it first partitions each core's schedule into segments that fit within hardware budgets. Then it performs deadlock-free on-chip reconfiguration at segment boundaries, exploiting the all-or-nothing completion semantics of collectives to minimize synchronization cost.

We implement MeshRT on a commodity wafer-scale accelerator and evaluate it on widely used AI models, including both MoE models such as GPT-OSS 120B [42], GPT-OSS 20B [43], Qwen3.5-35B [54], and dense models such as Qwen3.5-9B [55]. Compared to WaferLLM [22], the state-of-the-art wafer-scale software stack, MeshRT adds 25–40 μs (7.6–9.5%) per decode token in matched configurations while improving throughput by up to 56×. Against measured single-instance SGLang results on H200, MeshRT achieves 4.5–7.0× lower latency. Under ideal linear scaling of those GPU measurements to an area-normalized H200 cluster, MeshRT improves decode energy efficiency by 1.6–12.1× in the low-latency regime. We believe MeshRT marks an important step toward the coming era of low-latency AI, making such infrastructure substantially more affordable and broadly deployable.

2 Background and Motivation

2.1 Low-Latency Inference as a Premium Market

Low-latency inference has become a first-class design objective. Interactive coding assistants [15, 18, 44, 58], copilots [35], and agentic workloads [19, 21, 45, 48] all depend on fast per-token generation: lower decode latency directly reduces round-trip time in multi-turn interactions and enables tighter feedback loops. Providers have begun offering dedicated low-latency serving tiers [6, 9, 47], reflecting the fact that latency and throughput require fundamentally different system tradeoffs.

This demand is already reflected in production deployments. Anthropic offers a Fast mode for Claude Opus that runs at approximately 2.5× its default speed [9]. OpenAI serves Codex-Spark on Cerebras hardware at more than 1,000 tokens/s for real-time coding workloads [46] and previews an Ultrafast API tier serving GPT-5.6 Sol at up to 750 tokens/s, up to 14× faster than standard processing [47]. Once low latency is productized as a premium tier, reducing its serving cost becomes an economic imperative, and the key lever is achieving high throughput without sacrificing the low latency that defines the tier.

2.2 Wafer-Scale Chips for Low-Latency Inference

Wafer-scale accelerators (Figure 1) are a natural fit for low-latency inference because they can reduce the two dominant sources of per-token latency on conventional hardware [22]. First, latency-critical data can reside in on-chip SRAM when capacity permits, reducing the off-chip HBM accesses that bottleneck decode on GPUs, where each token generation requires reading the full KV cache and model weights through a memory bus orders of magnitude slower than SRAM. Second, all inter-core communication traverses an on-chip network-on-chip (NoC) fabric rather than PCIe, NVLink, or InfiniBand, eliminating the off-chip interconnect latency that grows with tensor-parallel degree in multi-GPU deployments. The Cerebras WSE-3 [11] exemplifies this architecture: it integrates approximately 900,000 processing elements (PEs) on a single wafer in a 762×1172 2D array, with 44 GB of aggregate on-chip SRAM and 21 PB/s of aggregate memory bandwidth. Each PE (hereafter referred to as a core) has approximately 48 KB of local SRAM, limited compute capability, and no shared memory; communication is point-to-point over the on-chip NoC connecting neighboring PEs [31].

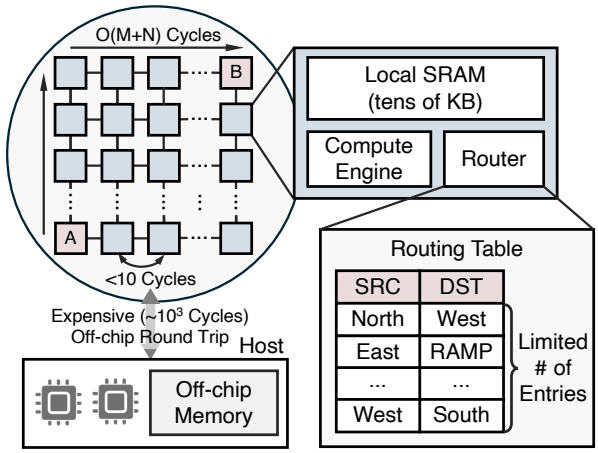


Figure 1. Wafer-scale accelerators as a distinct runtime setting. Host-mediated control incurs expensive round trips. Cores form a neighbor-only mesh of $M \times N$ cores, so chip-wide synchronization takes $O(M+N)$ hops and grows with physical distance. Each core exposes only bounded local resources, including local SRAM, finite routing-table state, and simple local branching rather than a general scheduler. These properties limit design choices.

Despite these latency advantages, the same architectural properties that reduce these bottlenecks also present the following challenges for runtime design.

Limited but programmable local control. A processing core can execute simple conditional branches based on local data, but it cannot run an open-ended runtime scheduler. This is fundamentally different from a GPU SM, where general-purpose control logic can execute on chip. On wafer-scale hardware, runtime handling must therefore be compiled into bounded local logic rather than delegated to a rich scheduler.

Expensive global coordination. A wafer-scale accelerator has no shared memory, no global cache, and no cheap centralized coordination point visible to all cores. Cores communicate only through the on-chip network, and communication cost grows with physical distance on the wafer. As a result, any runtime design that repeatedly consults a global scheduler, a shared request queue, or any other chip-wide synchronization will be costly.

Limited per-core resources to hold all execution states. Each core has only a small amount of local SRAM and a bounded amount of routing and queue state. This rules out two tempting approaches: storing large off-chip-style runtime metainfo structures on chip, and materializing every possible execution configuration in advance. A practical runtime must instead operate within tight local budgets and support reconfiguration when the full schedule does not fit.

Communication	Complexity	Cycle Cost
Broadcast	$O(N)$	$100 - 1000$
All-to-all	$O(N^2)$	$10^5 - 10^6$
Host synchronization	$O(1)$	7000

Table 1. Communication costs for runtime coordination on a wafer-scale accelerator.

These constraints create a runtime setting that is fundamentally different from conventional shared-memory architectures like GPUs and static accelerator pipelines [28, 49, 66]. Any runtime decision that GPU serving systems handle through a centralized host scheduler or shared memory must instead be resolved locally on each core, within tight resource budgets and without global coordination.

2.3 Runtime Dynamism Erodes Latency

Modern LLM serving workloads introduce various forms of *runtime dynamism*: aspects of the concrete execution path that cannot be fully determined by the precompiled schedule and must be resolved as inference proceeds. Without explicit runtime support for on-chip dynamism, existing wafer-scale systems are forced into one of two costly fallbacks: falling back to static execution (sacrificing resource efficiency) or offloading dynamic decisions to the host (sacrificing latency).

Scenario 1: Static execution is wasteful due to dynamic request sets. One important source of runtime dynamism is that the active request set evolves during execution. Continuous batching [66] is a canonical example: to keep throughput high, the system must refill slots vacated by completed requests and determine which request state remains relevant at each decode step. Similar effects arise in micro-batch pipeline execution [41, 60], dynamic batching [50], and speculative decoding with runtime acceptance decisions [30, 34].

On a wafer-scale chip, supporting this dynamism on chip presents a fundamental tension. One option is to propagate request-completion events to all cores so that each can perform local management, for example, evicting stale KV cache entries or reassigning batch slots. But the chip lacks on-chip inter-core coordination: the only available mechanism is a blind broadcast that must reach all ~900,000 cores, costing $O(N)$ hops and thousands of cycles per event (Table 1). As batch size is crucial for high throughput serving, these operations accumulate quickly and inflate per-step latency.

The alternative is to eliminate dynamism altogether by fixing the batch composition for the entire serving session, padding every request to the maximum sequence length so that no per-step reconfiguration is needed. This avoids broadcast overhead but trades it for wasted compute.

To quantify this waste, we measure the padding ratio, the fraction of decode compute spent on padded (zero-useful-work) tokens, under static batching across two representative workload distributions (ShareGPT [8] and OpenThoughts [20])

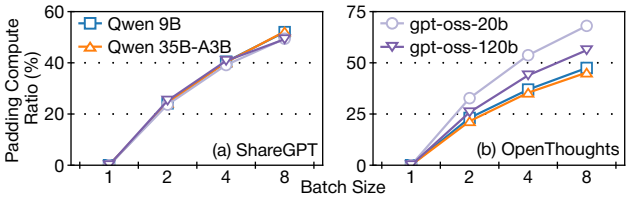


Figure 2. Padding compute ratios across different models increase with serving batch size.

and a mixture of dense and MoE models (Qwen3.5-Dense-9B, Qwen3.5-MoE-35B, GPT-OSS-MoE-20B, and GPT-OSS-MoE-120B). As depicted in Figure 2, at batch size 8, padding consumes 45–52% of compute on ShareGPT and 45–68% on OpenThoughts. The problem is both consistent and severe: it holds across dense and MoE architectures alike, and worsens with higher output-length variance (OpenThoughts exhibits longer reasoning chains than ShareGPT).

Scenario 2: Runtime decisions can stall the next step. A second source of runtime dynamism is that runtime results may determine what happens next, requiring coordinated reconfiguration of on-chip routing state. MoE serving with expert parallelism [17, 56] is a concrete example: the gating network produces per-token expert assignments only after the current activation is processed, and these assignments vary from token to token. Similar effects arise in conditional-depth models [57], token-dependent routing [5], and speculative decoding [30].

On a wafer-scale mesh, a layer's experts are partitioned across disjoint core groups so that the experts a step activates compute concurrently, each over a collective spanning far fewer cores. The group a token must reach is therefore known only after gating, and acting on this requires synchronized state changes: the attention-stage cores must communicate target-expert identifiers to the expert-stage cores so that both sides can configure routing entries for the correct data paths. Because these core groups may share relay links on the mesh, reconfiguration must be coordinated, installing a new routing entry while a neighbor still relies on the old one can corrupt in-flight data and cascade into deadlock. Without explicit on-chip support, the system faces two costly alternatives: all-to-all synchronization at $O(N^2)$ messages on a mesh with no multicast primitive (Table 1), or deferring each decision to the host at ~7,000 cycles per round trip (Table 1), 1.9-3.5× the on-chip decode step itself. Since a single token generation involves hundreds of such interactions, host-mediated control accounts for 65-78% of per-step latency in our measurements.

2.4 Existing Approaches Fall Short

Existing systems largely make one of two design choices, and neither is satisfactory for low-latency wafer-scale inference.

Approach 1: keep operator execution on chip, but keep scheduling logic off chip. This is the dominant choice in GPU serving systems [28, 66, 71]. Operators execute on the device, but request queues, dispatchers, and much of the runtime logic for handling dynamism remain on the host. This works well when the host can cheaply coordinate device execution and maintain large centralized runtime state.

On wafer-scale accelerators, however, this design choice conflicts with the hardware setting above. Repeated host intervention interrupts on-chip execution, incurs host-device round trips on latency-sensitive steps, and prevents the system from fully exploiting the benefit of keeping latency-sensitive control on chip. As a result, low latency becomes harder to maintain, and throughput suffers because runtime decisions cannot be absorbed cheaply where execution is happening.

Approach 2: keep both operator execution and schedule on chip through precompiled execution. This is the dominant choice in compiler-style wafer-scale and SRAM-first systems [1, 2, 22]: to preserve low latency, the system avoids off-chip runtime control by precompiling the execution schedule.

However, this design struggles with runtime dynamism. Continuous batching requires the active request set to evolve over time. MoE routing requires token-dependent decisions known only at runtime. Without an explicit mechanism for handling these decisions on chip, such systems either support these features poorly, restrict them heavily, or rely on expensive overprovisioning by precompiling many execution variants. On a wafer-scale accelerator, this extra state and routing cost is especially problematic because per-core resources are already tight. The result is a different but equally unsatisfactory outcome: low latency can be preserved, but throughput remains low and deployment cost remains high.

3 Design Intuitions

MeshRT keeps the inference control path on chip and extends compiled execution so that important runtime events can also be handled on chip. We call this new system architecture approach **compile-time governance of runtime dynamism** on wafer-scale accelerators.

Figure 3 illustrates this idea. MeshRT takes an optimized computation graph from an upstream compiler together with a specification of runtime dynamism (e.g., continuous batching, MoE routing) as input, and generates precompiled per-core schedules that interleave *data events* for executing inference with *dynamic control events* for handling runtime dynamism. These dynamic techniques are established inference optimizations, not contributions of this paper; what MeshRT contributes is making them applicable and efficient on a wafer-scale accelerator, where their usual implementations rely on a host runtime and shared memory that the hardware does not provide. Because both types of events are

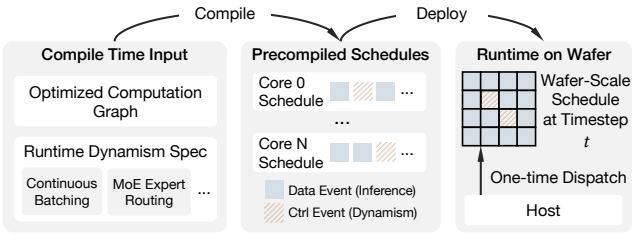


Figure 3. MeshRT design intuition: compile-time governance of runtime dynamism. Instead of deferring dynamic decisions to host control, MeshRT compiles them directly into per-core schedules as control events that execute alongside data events entirely on chip. Each core then runs its own schedule asynchronously, coordinating only through data arrival.

encoded at compile time, dynamic control remains on device: each core on the wafer-scale engine executes its schedule autonomously, host intervention is minimized, and the host only needs to install each per-core schedule once. The dynamism-aware precompiled schedule is then mapped and executed on the cores available on the wafer-scale accelerator. By doing so, MeshRT can preserve the low-latency benefits of wafer-scale accelerators while still supporting high-throughput features such as continuous batching and MoE routing.

Compile-time governance of runtime dynamism introduces two challenges to the wafer-scale system.

Challenge 1: Dynamism-aware schedule synthesis (§4.2). Because execution is compiled, every core needs a local schedule that specifies what to receive, compute, and send; collectively, all per-core schedules must faithfully realize the global computation graph. The input is a logical graph of kernel instances with global dependencies, not a set of local instruction streams. Bridging this gap already requires a non-trivial synthesis step; the additional requirement that runtime dynamism be absorbed *within* the synthesized schedule, rather than deferred to a separate runtime layer, makes the problem harder. The compiler must identify, at synthesis time, which cores receive data whose interpretation depends on a runtime decision and which cores produce results that introduce new runtime decisions for downstream cores, and it must weave the corresponding control logic into the per-core schedule.

Challenge 2: Safe incremental materialization (§4.3). To execute on chip without host intervention, schedules must fit within the limited memory, routing, and queue resources of each core. A core typically participates in many kernels across a forward pass, and the union of their resource footprints can exceed what the core can instantiate at once. On a GPU this is resolved by spilling to HBM or rescheduling at runtime; here, neither option is available. MeshRT must partition the schedule into segments and compile the segments as well as their transitions into deadlock-free on-chip execution, a common challenge in large-scale asynchronous distributed runtimes.

Both challenges rely on a shared on-chip control substrate (§4.2, §5.1). Each core maintains a small *local control state*, such as which request slots are active or which routing result applies next, and uses bounded *control messages* exchanged over the same on-chip network. For schedule synthesis, this substrate is what allows dynamism-handling logic to be woven into the compiled schedule: when the next data path is already valid, control metainfo rides alongside ordinary activations; when the path depends on a runtime result, a short control-message exchange resolves it within the affected region before data movement proceeds. For incremental materialization, the same substrate carries reconfiguration metainfo at segment boundaries, enabling cores to transition safely between materialized segments without host coordination.

Two properties make this substrate viable under the constraints of §2.2. First, control messages are bounded: a few bytes of metainfo per batch slot, piggybacked on activations three orders of magnitude larger, so dynamism handling costs no separate network traversal. Second, participation is local: because the compiler already knows which cores a decision affects, only those cores exchange control messages, avoiding both the $O(N)$ broadcast and the host round trip priced in Table 1.

4 System Design

Figure 4 gives an overview of MeshRT. Given a computation graph, MeshRT synthesizes *dynamism-aware per-core schedules* that interleave data-processing and dynamism-handling sub-schedules in a single compilation pass (§4.2). It then makes these schedules executable under tight per-core resource budgets through *safe incremental materialization* (§4.3).

4.1 System Model and Design Assumptions

MeshRT sits between an upstream compiler and the vendor-provided hardware primitives. Its input is a *compiler-optimized computation graph* - a logical description of the inference workload. Additionally, MeshRT requires *mapping choices* that assign kernels, i.e., nodes in the computation graph, to spatial regions on the wafer.

A *kernel instance* denotes a concrete occurrence of a kernel in the model's logical execution; each is associated with a kernel function that specifies its computation pattern and a mapping choice R_k that assigns it to a spatial region on the wafer. The input to MeshRT is therefore a computation graph $G = (K, E)$ together with mapping choices $M = \{R_k \mid k \in K\}$, where K is the set of kernel instances and E is the set of logical dependencies between them. Edges in E encode execution order: a kernel instance executes only after all of

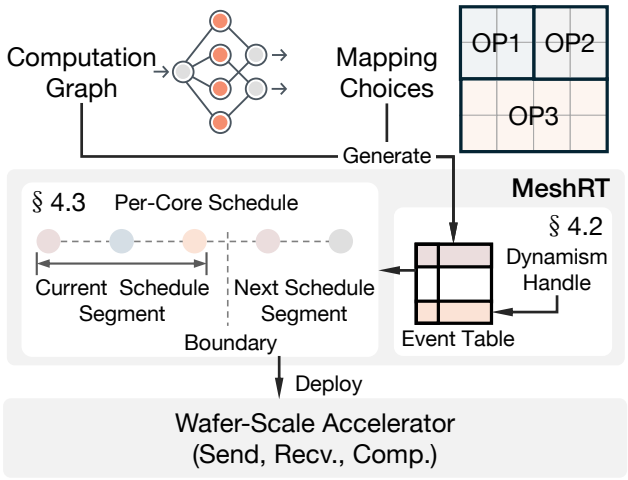


Figure 4. MeshRT system overview. Given a computation graph and mapping choices, MeshRT synthesizes dynamism-aware per-core event tables (§4.2), partitions them into segments that fit hardware resource limits (§4.3), and deploys the result onto the wafer-scale accelerator.

its predecessors have produced their outputs. Together, G and M define the mapped computation that MeshRT lowers to per-core execution.

The hardware exposes a per-core primitive interface $\Pi = \{\texttt{Compute}, \texttt{Receive}, \texttt{Send}\}$. `Compute` applies a local operation to core-resident data, including arithmetic computation, conditional branching, and local hardware reconfiguration (such as rebinding routing color IDs and buffer queue assignments on the WSE-3); `Receive` and `Send` move data between neighboring cores. MeshRT turns the mapped computation into per-core compiled schedules over this interface.

Note that mapping choices are important: prior work has explored mapping strategies for spatial accelerators [26, 27, 74]. Such choices are outside our scope; we select them empirically for the evaluated models. Our focus is on the system design that follows once such choices are available.

4.2 Dynamism-Aware Per-Core Schedule Synthesis

To synthesize per-core schedules that support runtime dynamism, our key observation is: although the specific values of inference runtime decisions are unknown until execution, the control *structure* of a per-core schedule, including where dynamism arises, what type it is, and how each core should handle it, can be fully determined at compile time. LLM execution uses a set of *known* kernel types. Once the graph and mapping are fixed, each kernel determines where dynamism may arise and which cores may consume or produce it. MeshRT captures this with two predicates, ReceivesDynamism(k, c) and ProducesDynamism(k, c). Whenever either holds, the compiler injects metainfo-handling sub-schedules around the kernel's core schedule. The injection is bounded: it applies only to the cores in R_k, the region the

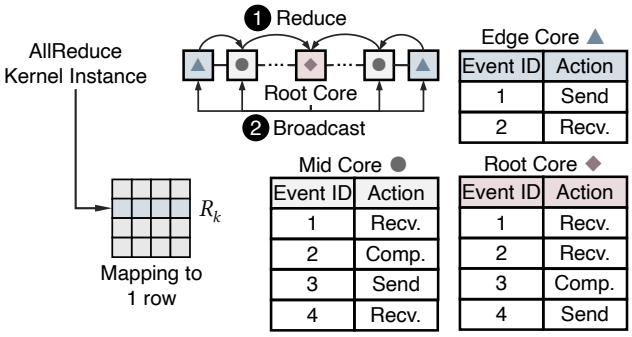


Figure 5. Per-core schedule synthesis for an AllReduce kernel mapped to one row of cores. The kernel function and mapping determine role-specific event-action tables: edge cores receive a partial sum, accumulate locally, forward it, and later receive the broadcast result; mid cores relay and accumulate; and the root reduces and broadcasts.

kernel is mapped to, so dynamism handling stays within the cores a decision actually reaches and never becomes chip-wide. In this way, MeshRT turns runtime dynamism from an external runtime responsibility into a compile-time scheduling object. Per-core resource limits are handled separately by incremental materialization (§4.3).

This synthesis proceeds as shown in Algorithm 1. (1) **Initialization (line 1–2):** Allocate an empty schedule S_c for each core that participates in at least one kernel instance. (2) **Topological traversal (line 3–13):** Walk kernel instances in spatial-temporal dependency order so that each kernel is processed only after its predecessors are known. (3) **Incoming dynamism resolution (line 6–8):** For each core c in the kernel's assigned region R_k, the compiler checks whether c receives runtime dynamism in kernel k (ReceivesDynamism). If so, a metainfo resolution sub-schedule is inserted: the core first receives and parses incoming metainfo to resolve the dynamism before computation begins. (4) **Kernel lowering (line 9–10):** The kernel's own core schedule is generated via GetCoreSche, determined by the kernel function f_k, the assigned region R_k, and the primitive interface Π. (5) **Outgoing dynamism production (line 11–13):** The compiler checks whether c produces runtime dynamism in kernel k (ProducesDynamism). If so, a metainfo production sub-schedule is appended: the core generates metainfo from its computation result and sends it ahead of the data to downstream cores.

All sub-schedules are attached to S_c via AppendSche together with the predecessor guard $p_k = \text{Pred}(k)$. Iterating in topological order preserves the spatial-temporal dependencies of G, while iterating over $c \in R_k$ specializes the schedule to the spatial mapping M.

Figure 5 illustrates a synthesis example for `AllReduce`. Consider an `AllReduce` kernel instance k whose kernel function specifies a reduction over a set of cores, and whose

Algorithm 1: Dynamism-aware Per-core Schedule

Input: $G = (K, E)$; $M = \{R_k \mid k \in K\}$; Π
Output: $\{S_c\}$

```
foreach Core c ∈ ⋃_{k∈K} R_k do
    S_c ← [ ];
foreach k ∈ TopologicalOrder(G) do
    p_k ← Pred(k);
    foreach c ∈ R_k do
        if ReceivesDynamism(k, c) then
            s_recv ← GetMetaResolver(k, c);
            AppendSche(S_c, s_recv, p_k);
        s_{k,c} ← GetCoreSche(k, c);
        AppendSche(S_c, s_{k,c}, p_k);
        if ProducesDynamism(k, c) then
            s_send ← GetMetaProducer(k, c);
            AppendSche(S_c, s_send, p_k);
return {S_c};
```

mapping choice assigns it to a region R_k of four cores arranged in a line. The compiler walks the cores in R_k and calls GetCoreSche(k, c) for each. The kernel function describes each core's role in both phases of the collective, so different cores receive different event-action tables. An edge core receives a partial sum, accumulates its local contribution, forwards the new partial sum toward the root, and later receives the broadcast result. A mid core relays and accumulates during reduction and relays the result in the reverse direction. The root receives the reduction contributions, computes the final result, and initiates the broadcast. The compiler also evaluates ReceivesDynamism(k, c) and ProducesDynamism(k, c) for each core; since `AllReduce`'s kernel function involves no runtime-dependent metainfo, both checks return false, and no metainfo sub-schedules are inserted. The result is a per-core schedule consisting purely of data-processing entries, the baseline case of Algorithm 1 where only step (4) applies.

Use case: continuous batching The active request set evolves during execution: requests complete at different decode steps and new requests must refill vacated slots, which batch slots are active cannot be fixed at compile time (Figure 6). The sampling kernel at the final transformer layer satisfies ProducesDynamism: after computing output tokens, each core detects end-of-sequence events, generates batch-state metainfo encoding which slots have completed, and sends it ahead of data to the first layer of the next decode step. The first transformer layer satisfies ReceivesDynamism: each core parses this metainfo to learn which slots are active before computation begins. Request arrivals and completions are thus absorbed entirely on chip.

Use case: MoE. Expert-parallel mappings, which MeshRT takes as input (§4.1), place a layer's experts on disjoint core regions so that the experts a step activates compute concurrently. We call each spatial core region assigned to one expert or a fixed expert group an *expert region*. The gating network selects a token's destination region only at runtime, so the attention-to-expert data paths cannot be fixed at compile time (Figure 7). The gating kernel from the attention stage satisfies ProducesDynamism: after computing top-k expert selections, each core generates routing metainfo and prepends it to the token data sent toward the expert region. Relay cores along the path and the expert FFN kernel both satisfy ReceivesDynamism: relay cores parse routing metainfo to determine forwarding directions, and expert cores parse it to identify which tokens to process. Token-dependent routing is thus resolved hop-by-hop on chip: metainfo of compile-time-known size travels over compile-time-known candidate paths, establishing the MoE dispatch and combine routes at runtime.

Generality beyond the two examples. Continuous batching and MoE routing are the two use cases we implement and evaluate, but the ReceivesDynamism/ProducesDynamism mechanism is general. A runtime decision fits the same pattern when its producing kernels emit bounded metainfo and downstream consumers use it to determine their next actions. For global hot-expert load balancing, local load-statistic kernels produce metainfo, a collective decision kernel receives these statistics and produces a load-balancing decision, and the affected routing and expert kernels receive that decision. Speculative decoding is one example: the verification kernel produces an acceptance decision (how many draft tokens to keep), and downstream kernels receive this decision to determine which token states to carry forward. Sparse or adaptive attention is another: a selection kernel produces a set of active KV entries, and the attention kernel receives this set to determine which entries to fetch and compute over. In each case, the compiler evaluates ReceivesDynamism and ProducesDynamism for every mapped kernel and core, and inserts the corresponding metainfo resolver and producer sub-schedules; no change to the synthesis algorithm is needed.

4.3 Safe Incremental Schedule Materialization

Once MeshRT has a precompiled schedule that includes both data-processing logic and event-driven logic for handling runtime dynamism (§4.2), the remaining problem is executing this schedule on cores with tight hardware budgets like routing resources. Our key idea is to partition the schedule into *schedule segments* whose peak resource usage fits within hardware limits, and to materialize these segments incrementally at runtime.

Incremental materialization must rely entirely on local, asynchronous on-chip message exchange; otherwise, cores would stall frequently for synchronized segment reconfiguration, increasing per-step latency and reducing throughput.

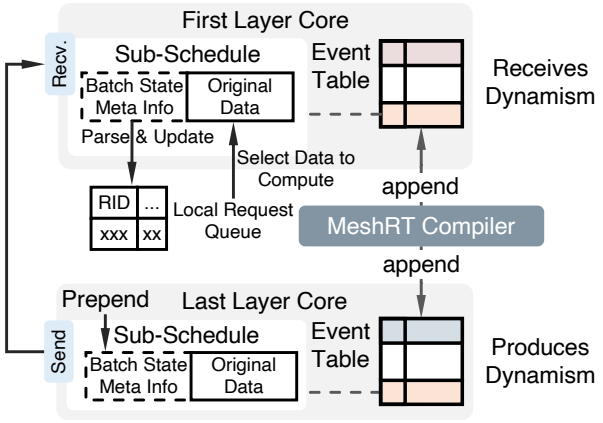


Figure 6. Synthesized per-core sub-schedules for continuous batching. The last-layer core prepends batch-state metainfo to outgoing data; the first-layer core parses it and updates its local request table to select active slots.

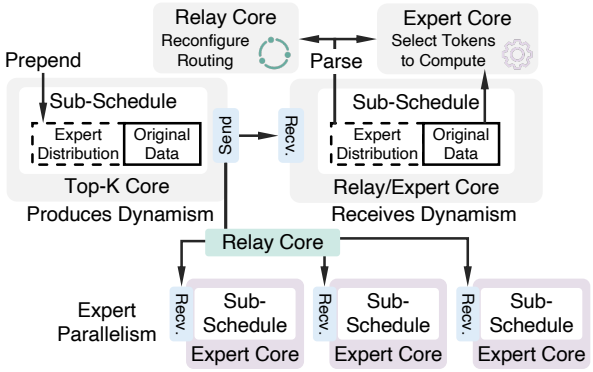


Figure 7. Synthesized per-core sub-schedules for MoE routing. The top-k core prepends expert-distribution metainfo to token data; downstream relay cores parse it to reconfigure routing, and expert cores parse it to select which tokens to compute.

At the same time, asynchronous reconfiguration must be *safe*: switching segments without coordination can leave neighboring cores in inconsistent states, leading to deadlock.

We introduce *safe incremental schedule materialization*, which prevents reconfiguration-induced deadlock. It reconfigures segments through local asynchronous event processing and message passing, without global synchronization. We describe its key steps below and omit its formal declaration due to the page limit:

❶ **Resource initialization.** For each core c, initialize resource budget $R \leftarrow \mathrm{Cap}(c)$ representing the core's full hardware capacity, including routing channels, queue bindings, buffers, and local control-state storage.

❷ **Continuous segment extension.** Walk the sub-schedules in S_c in order. As long as the next sub-schedule fits within R, append it to the current segment and deduct its resource demand from R.

❸ **Overflow and boundary classification.** When the next sub-schedule exceeds R, the current segment must be closed. The compiler examines the most recent kernel in which core c participated: if that kernel's function already guarantees that all participating cores reach a consistent safe state upon completion (as collectives such as `AllReduce` naturally do), the boundary is safe and no extra coordination is needed.

❹ **Closing the segment.** At a safe boundary, the compiler appends a reconfiguration sub-schedule to close the current segment. If the boundary is not inherently safe, the compiler first inserts a local synchronization step among the participating cores to establish a consistent state, and then appends the reconfiguration.

❺ **Resource reset and new segment.** After reconfiguration releases the old bindings, reset $R \leftarrow \mathrm{Cap}(c)$ and start a new segment beginning with the sub-schedule that triggered the overflow.

Why collectives provide natural safe boundaries. The key property is that a collective such as `AllReduce` has an *all-or-nothing completion* semantics: a core can only finish its local part of the collective after it has both contributed its own data and received the final result, which in turn requires every other participating core to have contributed. The collective's completion events therefore provide a consistent cut over its participating region. When the compiler selects this point as a segment boundary, MeshRT reuses that cut without additional coordination.

Why incremental schedule reconfiguration is safe. We view a segment transition as a distributed snapshot: a consistent cut, rather than a simultaneous global barrier [12]. MeshRT establishes this cut in two ways. When a segment ends at a collective, the collective's completion semantics already provide a consistent cut over its participating region. Otherwise, the compiler inserts a bounded marker exchange within the compile-time-known affected region, and each core switches when it reaches its local cut. Both cases preserve the same invariants: the cut covers every route affected by the reconfiguration, old-segment traffic remains before the cut, and new-segment traffic begins after it. Therefore, no message can observe mixed old and new bindings, preventing reconfiguration-induced deadlock without chip-wide synchronization.

5 System Implementation

We implement MeshRT on a Cerebras CS-3 system powered by the WSE-3 processor. Starting from a compiler-optimized computation graph and mapping choices, MeshRT synthesizes dynamism-aware per-core schedules (§4.2), partitions them via incremental materialization (§4.3), and installs the resulting per-core executables onto the wafer once. Figure 8

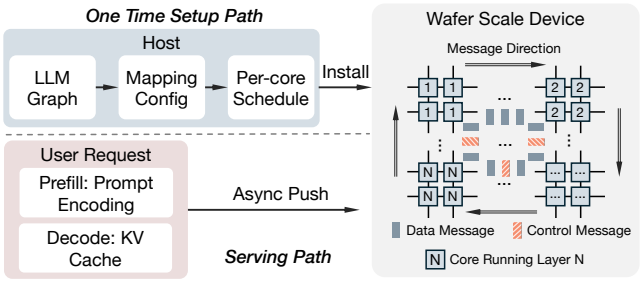


Figure 8. MeshRT system boundary. The host compiles and installs per-core schedules once (top left). During serving (bottom left), it asynchronously pushes prefill contexts and decode KV caches. All per-step execution, continuous batching, and dynamism resolution proceed on chip without host involvement (right).

shows the resulting system boundary: the host performs one-time compilation and installation; during serving, it only pushes request data asynchronously. All per-step scheduling, batch management, and dynamism resolution proceed entirely on chip.

5.1 Serving Data Path

Prefill processes prompts in chunks to reduce peak SRAM usage [3]. Decode is where continuous batching operates. Each core maintains a *local waiting request queue*, into which the host asynchronously pushes decode requests as (`request_id`, KV cache) or prefill requests as (`request_id`, prompt token IDs), without interrupting on-chip execution, as well as a *Request Descriptor Table (RDT)* that records the current batch composition locally—including request identifiers, KV cache pointers, sequence lengths, and other per-request metainfo. Micro-batches flow between pipeline stages carrying activation data and a compact piggybacked *control-flow (CF) payload*: only (`request_id`, `status`) per batch slot, typically a few bytes per request versus ~4 KB of activation data (<0.2% overhead).

A decode step in action. Consider slot 2 of a decoding micro-batch carrying request 17. While decode step t executes, the host may asynchronously enqueue request 42 without entering the per-token control path. At the final stage, on-chip sampling produces the next token; if request 17 emits an end-of-sequence token, the stage sets CF[2] = (17, `finished`). At step t+1, the first stage consumes this entry, releases request 17's KV slot, admits request 42 from its waiting queue, replaces RDT[2] = $(17, kv_{17}, \ell_{17})$ with $(42, kv_{42}, \ell_{42})$, and emits CF[2] = (42, `active`). Each following stage receives the activation and CF together, updates its local RDT, executes the ordinary layer events, and forwards both; sampled tokens in continuing slots feed back on chip to start the next autoregressive step. In the per-core event table, this adds one metainfo-resolution `Compute` before each affected layer and one metainfo-production `Compute` after sampling. Because CF is piggybacked on existing activation transfers, it adds no standalone `Send` or `Receive`. These events remain in the current materialized segment; segment transitions still occur only at the precompiled safe boundaries described in §4.3.

5.2 Multi-wafer Execution

Our multi-wafer layouts partition the model only along the layer dimension: each wafer holds a contiguous stack of decoder layers and executes as one or more pipeline stages. We do not split an attention or expert layer across wafers, so KV state, expert routing, and schedule reconfiguration remain wafer-local. Consequently, the only bulk cross-wafer transfer is the hidden state at a pipeline-stage boundary; the CF payload rides with this hidden state and introduces no separate control round trip. Cerebras reports 1.2 Tbps of inter-system bandwidth for CS-3 [32]. In our measurements, these handoffs use approximately 100 Gbps, less than 10% of that bandwidth, and add less than 1% to end-to-end decode latency.

5.3 Safe Reconfiguration on GALS Hardware

The WSE-3 is a globally asynchronous, locally synchronous (GALS) system: each core runs on its own clock and coordinates solely through data arrival. A core whose output queue has drained has reached *local* quiescence, but upstream cores may still have data in transit through its router. Reconfiguring routing entries in this state corrupts in-flight data and causes deadlock. These races rarely surface in simulation, where the scale of parallelism is too small to expose meaningful asynchrony, and they manifest only at full hardware scale. Counterintuitively, low-latency inference maximizes this risk. Fine-grained parallelism means small per-core data volumes, which drain queues long before upstream cores finish, widening the race window. Large data transfers trigger backpressure that implicitly aligns cores; the minimal-data, maximal-parallelism regime MeshRT targets is the most exposed.

MeshRT applies two strategies from §4.3. Most segment boundaries reuse the consistent cut already provided by a collective. For boundaries without such a natural cut, the expert-dispatch and combine sites of each MoE layer, MeshRT synthesizes bounded marker-based synchronization only within the affected region. The sequence of reconfiguration sites is fixed at compile time and repeats with each decode step; every transition remains local to one wafer, and pipeline-stage handoffs trigger no cross-wafer reconfiguration or synchronization. All reconfiguration costs are included in the reported device measurements.

6 Evaluation

We evaluate MeshRT against WaferLLM [22], the state-of-the-art in wafer-scale LLM inference systems, across four

models spanning Dense and MoE architectures. Our evaluation answers three questions:

❶ Does MeshRT's compile-time governance framework improve end-to-end inference throughput over static dataflow with branching? (§6.2)
❷ How does MeshRT compare to GPU-based serving in latency, throughput, and energy efficiency? (§6.3)
❸ What is the runtime cost of compile-time governance, and how do the serving policies it enables contribute to performance? (§6.4)

6.1 Experimental Setup

Testbed. Our wafer-scale testbed uses the Cerebras CS-3 system, powered by the WSE-3 processor: 900,000 compute cores, 44 GB on-chip SRAM, 21 PB/s memory bandwidth, and 214 Pb/s core-to-core fabric bandwidth [11]. The WSE-3 processor operates within a 15 kW power envelope [63]. For GPU comparisons, we use NVIDIA H200 SXM GPUs (up to 700 W TDP per card) [39] with SGLang 0.5.9 [59] as the serving engine. For chip-area normalization, we use 56 H200 dies per WSE-3: the WSE-3 occupies 46,225 mm^2, while the GH100 die used by H200 occupies 814 mm^2 [11, 38]. We measure the best-performing single data-parallelism instance on GPU (sweeping tensor-parallel degree), then linearly scale to the GPU count that matches the total WSE-3 silicon area used by each model configuration.

Models. We evaluate four models that span the dense–MoE spectrum and exercise different dimensions of runtime dynamism:

- **Qwen3.5-Dense-9B** [55]: Dense transformer (9B parameters, FP16). Exercises continuous batching and pipeline parallelism but no expert routing. Runs on 2×WSE-3.
- **Qwen3.5-MoE-35B** [54]: 35B-total, 3B-active MoE with 256 experts (top-8 + 1 shared), FP8 quantization. Requires a 6×WSE-3 placement.
- **GPT-OSS-MoE-20B** [43]: 20B-total, 3.6B-active MoE with 32 experts (top-4), MXFP4 quantization. Maps to 4×WSE-3.
- **GPT-OSS-MoE-120B** [42]: 120B-total, 5.1B-active MoE with 128 experts (top-4), MXFP4 quantization. Requires an 8×WSE-3 placement.

Wafer-scale measurements. For Qwen3.5-9B and GPT-OSS-20B, we measure the model-specific components needed by their complete 2- and 4-WSE-3 configurations, respectively, and compose full-model throughput from the measured device TSC cycles. These deployments span multiple wafers despite WSE-3's 44 GB aggregate SRAM because the memory is distributed across PEs and each PE retains a complete compiled program image, even with staged schedule materialization, so that local sub-state machines can be activated at low overhead; the resulting replicated code footprint consumes substantial SRAM on every wafer. Keeping the program image resident does not eliminate the need for incremental materialization: only the active schedule segment instantiates its routing and queue state, since these per-core hardware resources remain scarce. For Qwen3.5-35B and GPT-OSS-120B, whose full placements span 6 and 8 WSE-3s, we measure their model-specific and transformer layer components on a two-wafer setup and project full-model results by composing the measured device TSC cycles according to each model's layer placement and pipeline-stage bottleneck. Unless stated otherwise, the reported 35B and 120B results use these projections. The area-equivalent GPU cluster results are likewise projections: we measure the best-performing single data-parallel instance for each model and scale it ideally to the corresponding GPU count rather than measuring a physical multi-GPU cluster.

Baseline. We compare against WaferLLM [22], the published state of the art for LLM inference on the Cerebras WSE. WaferLLM uses PE-level if-else branching to select among pre-compiled execution paths, enabling multi-layer packing within a single PE block. However, it lacks three capabilities that MeshRT provides: (1) per-request metainfo propagation (no continuous batching, batches are fixed with padding to uniform batch size), (2) distributed routing resolution (no cross-block expert routing, all MoE experts are spatially tiled in a single PE block, forfeiting expert parallelism), and (3) decentralized PE reconfiguration.

WaferLLM represents the best strategy achievable with PE-level branching alone, without a systematic governance framework built atop it. Both systems share the same static execution foundation (2D-tiled GEMM/GEMV, AllReduce via neighbor-based reduction trees); all differences stem from how each system handles runtime dynamism.

Metrics. We report: (1) single-user throughput (tokens/s), the generation speed experienced by one user; (2) system throughput (tokens/s), aggregate tokens produced across all concurrent requests; and (3) energy efficiency (tokens/joule), system throughput divided by total power draw.

Batching configurations. For MeshRT on WSE-3, `micro-bs` denotes the number of requests processed together within a pipeline stage. Models use different numbers of wafers and spatial mappings, resulting in different pipeline stage counts, which may also change with `micro-bs`. We therefore sweep a common set of micro-batch sizes within each inference phase. The total serving batch size is the in-flight request concurrency c, reported in Figure 9. Under the evaluated pipeline configurations, c = `micro-bs` × stages.

6.2 End-to-End Performance

This experiment compares the single-user and system throughput Pareto frontiers of MeshRT and WaferLLM across all four models.

Prefill experiments. At 4K context, the two systems have nearly identical single-user throughput at each micro batch.

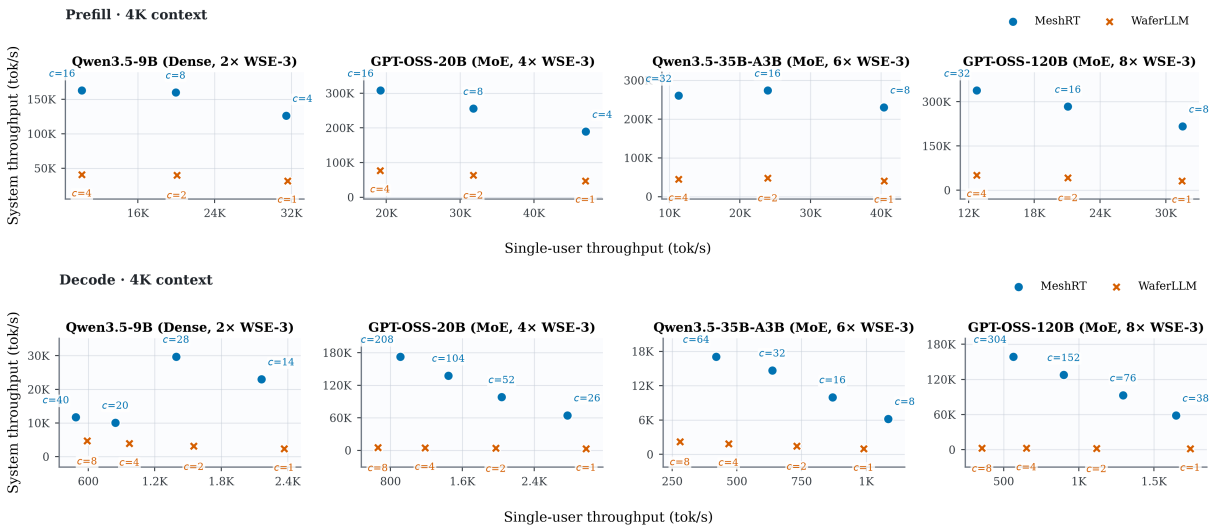


Figure 9. System throughput vs. single-user throughput on WSE-3 at 4K context for prefill (top) and decode (bottom). Comparing each system's peak over the evaluated micro-batches, MeshRT improves system throughput by 4.0–6.6× during prefill and by up to 55.8× during decode. Decode points do not apply workload-level padding correction.

Comparing the highest-system-throughput point of each system over the evaluated micro-batches, MeshRT improves system throughput by 4.0× on Qwen3.5-9B and GPT-OSS-20B, 5.7× on Qwen3.5-35B, and 6.6× on GPT-OSS-120B. The current MoE prefill variants use the same single-block FFN mapping as their static baselines. Table 4 separately decomposes a controlled path from the static `bs=1` configuration to pipelined MeshRT at `micro-bs=1` and then to `micro-bs=4`; its factors therefore use different endpoints from these peak-to-peak ratios.

Decode experiments. At the highest-system-throughput point for each model in Figure 9, MeshRT improves over WaferLLM by 6.3× on Qwen3.5-9B, 32.7× on GPT-OSS-20B, 7.6× on Qwen3.5-35B, and 55.8× on GPT-OSS-120B. These gains come from overlapping micro-batches across the compiled pipeline and, for MoE models, executing experts in parallel. The plotted values do not include workload-level padding correction; continuous batching's additional contribution is quantified in Table 4. The compact Qwen3.5-35B projection reported in Tables 2 and 3 is not part of this common-layout Pareto sweep.

Governance overhead. Figure 10(a) compares per-user throughput for the two models with matched MeshRT and WaferLLM configurations. At 4K context and micro-batch size 1, MeshRT delivers 2,163 and 2,770 tok/s per user on Qwen3.5-9B and GPT-OSS-20B, respectively, versus 2,368 and 2,980 tok/s per user for WaferLLM. Equivalently, on-chip governance adds only 40 μs (9.5%) and 25 μs (7.6%) per token. In particular, the larger GPT-OSS-20B does not incur a larger penalty, showing that this overhead is determined by the fixed metainfo handling rather than model size. Because CF

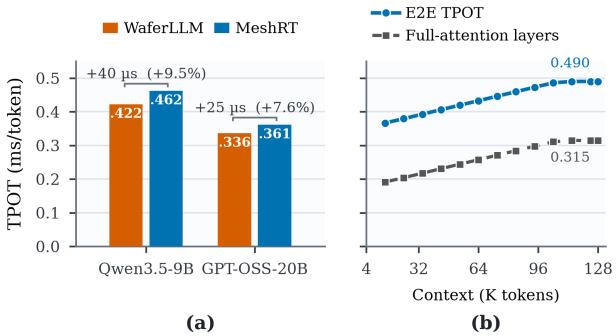


Figure 10. Decode TPOT at micro-batch size 1 (lower is better). (a) At 4K context, under matched configurations, MeshRT adds only 40 μs (9.5%) on Qwen3.5-9B and 25 μs (7.6%) on GPT-OSS-20B relative to WaferLLM. (b) GPT-OSS-20B long-context decode, showing E2E TPOT and the time spent in full-attention layers as the context grows to 128K.

metainfo is piggybacked on activations, governance adds no standalone send or receive; continuous batching therefore increases throughput without adding a separate communication step to the per-token path.

Figure 10(b) further shows that per-user throughput decreases gradually as the context grows toward 128K because full-attention work increases. Beyond approximately 103K context, new KV entries are transferred to the FFN PEs and processed in parallel with the fixed 431-slot attention prefix on the ATTN PEs. Since the FFN-side work has not yet overtaken the ATTN-side critical path, per-user throughput remains nearly flat thereafter.

Spatial batching tradeoff. Qwen3.5-9B exposes a wafer-scale-specific non-monotonicity. At `micro-bs=2`, each layer uses a 256×256-PE region, so the two WSE-3s fit 12 layer regions; including the head and tail yields 14 pipeline stages. At `micro-bs=4` and 8, each layer instead requires a 512×512-PE region, leaving room for only three layer regions and thus five stages including the head and tail. The corresponding in-flight concurrencies, c = `micro-bs` × stages, are $2 \times 14 = 28$, $4 \times 5 = 20$, and $8 \times 5 = 40$. System throughput is not this product: it is `micro-bs` divided by the bottleneck-stage time. That time rises from 74K cycles at `micro-bs=2` to 436K and 752K cycles at `micro-bs=4` and 8, yielding 29.7K, 10.1K, and 11.7K tok/s, respectively. Thus, the slower, coarser stages outweigh the benefit of a larger micro batch.

6.3 Comparison with GPU Serving

We compare MeshRT on WSE-3 against SGLang on H200 GPUs under latency-oriented and throughput-oriented policies for both prefill and decode (Table 2). Prefill uses 4K context length. GPU system throughput assumes ideal linear scale-out to the area-equivalent GPU count and is therefore an optimistic upper bound.

Decode: MeshRT excels in per-user latency and energy. For latency-oriented decode (`micro-bs=1`), MeshRT improves per-user throughput by 4.5–7.0× and energy efficiency by 1.6–12.1×; system throughput ranges from 0.61–4.6× of the GPU projection. GPU scale-out raises aggregate throughput but does not improve the per-user speed of each data-parallel instance. MeshRT's per-user advantage comes from the WSE-3's 21 PB/s SRAM bandwidth and from on-chip dynamism resolution via CF payloads and RDTs (§5.1), which avoids per-step host–device coordination.

For throughput-oriented decode (Table 2b), MeshRT retains a 3.7–8.4× per-user advantage, while the area-equivalent GPU cluster delivers 1.2–4.9× higher aggregate throughput across the four models under ideal linear scaling. MeshRT remains 1.1–2.1× more energy-efficient on Qwen3.5-9B, GPT-OSS-20B, and GPT-OSS-120B, while reaching 0.54× on Qwen3.5-35B. These operating points expose the tradeoff: GPU replicas maximize aggregate throughput, whereas MeshRT prioritizes request-level decode speed.

Prefill: structural mismatch, mitigated in practice. Prefill is compute-bound (GEMM), and here the WSE-3's architecture is at a structural disadvantage: its 900,000 simple cores trade compute density for aggregate SRAM bandwidth, whereas the H200's tensor cores are purpose-built for high-FLOPS matrix multiplication. For latency-oriented prefill, GPU per-user throughput is 1.3–2.3× higher; under throughput-oriented operation, projected GPU system throughput exceeds MeshRT by 25–61× under ideal linear scaling.

Pipeline parallelism partially compensates at the request level. At `micro-bs=4`, overlapping requests across pipeline stages allows MeshRT to match or slightly exceed GPU per-user throughput on Qwen3.5-9B, GPT-OSS-20B, and GPT-OSS-120B (1.0–1.1×). Qwen3.5-35B reaches 0.51× because its per-user throughput falls beyond `micro-bs=2`. The aggregate-throughput gap remains large for all four models.

		Prefill			Decode		
Model		**User**	**Sys**	**tok/J**	**User**	**Sys**	**tok/J**
Qwen3.5 -9B	GPU	72,757	2,037K	25.98	355	4,966	0.063
	MeshRT	31,448	126K	4.21	2,163	22,990	0.766
	Ratio	0.43×	0.06×	0.16×	6.1×	4.6×	12.1×
GPT-OSS -20B	GPU	80,264	8,990K	57.33	393	22,006	0.140
	MeshRT	47,018	190K	3.16	2,770	64,622	1.077
	Ratio	0.59×	0.02×	0.06×	7.0×	2.9×	7.7×
Qwen3.5 -35B-A3B	GPU	53,745	4,515K	19.19	243	10,191	0.043
	MeshRT	40,389	231K	2.56	1,085	6,207	0.069
	Ratio	0.75×	0.05×	0.13×	4.5×	0.61×	1.6×
GPT-OSS -120B	GPU	56,473	6,325K	20.17	276	15,465	0.049
	MeshRT	31,518	217K	1.81	1,647	58,305	0.486
	Ratio	0.56×	0.03×	0.09×	6.0×	3.8×	9.9×

(a) Latency-oriented (prefill `micro-bs=1`; decode `micro-bs=1`). GPU uses each model's latency-optimal `tp`.

		Prefill			Decode		
Model		**User**	**Sys**	**tok/J**	**User**	**Sys**	**tok/J**
Qwen3.5 -9B	GPU	9,167	4,107K	52.4	165	36,940	0.47
	MeshRT	10,161	163K	5.43	1,393	29,686	0.99
	Ratio	1.1×	0.04×	0.10×	8.4×	0.80×	2.1×
GPT-OSS -20B	GPU	18,499	16,575K	105.7	193	345,670	2.20
	MeshRT	19,188	308K	5.14	910	172,614	2.88
	Ratio	1.0×	0.02×	0.05×	4.7×	0.50×	1.3×
Qwen3.5 -35B-A3B	GPU	22,016	14,795K	62.9	114	307,685	1.31
	MeshRT	11,289	261K	2.90	419	63,274	0.70
	Ratio	0.51×	0.02×	0.05×	3.7×	0.21×	0.54×
GPT-OSS -120B	GPU	11,523	20,649K	65.8	107	385,101	1.23
	MeshRT	12,753	338K	2.82	564	158,701	1.32
	Ratio	1.1×	0.02×	0.04×	5.3×	0.41×	1.1×

(b) Throughput-oriented (prefill `micro-bs=4`; decode `micro-bs=8`, but 2 for 9B). GPU system throughput projected by linearly scaling `tp=1` (upper bound).

Table 2. MeshRT vs. GPU (H200) across prefill and decode. Micro-batch sizes refer to the MeshRT configurations on WSE-3. Qwen3.5-9B uses a smaller decode micro-batch because it achieves the highest system throughput among the evaluated configurations. **User** and **Sys** denote per-user and aggregate throughput. Each block reports GPU, MeshRT, and the MeshRT/GPU ratio (green ≥1×, red <1×). GPU system throughput is projected to the chip-area-equivalent GPU count. The 35B and 120B MeshRT results are projected as described in §6.1. Energy uses rated power: 15 kW/WSE-3 and 700 W/H200. All throughputs in tok/s; energy in tok/J.

	Single-User (tok/s)			System (tok/s)		
Model	WaferLLM	MeshRT	Ratio	WaferLLM	MeshRT	Ratio
Qwen3.5-9B	2,368	2,163	0.91×	4,717	29,686	6.3×
GPT-OSS-20B	2,980	2,770	0.93×	5,272	172,614	32.7×
Qwen3.5-35B-A3B	990	1,085	1.10×	2,238	63,274	28.3×
GPT-OSS-120B	1,742	1,647	0.95×	2,843	158,701	55.8×

Table 3. Matched-configuration per-step throughput and peak system throughput relative to static WaferLLM. Single-user throughput uses `micro-bs=1` for MeshRT and `bs=1` for WaferLLM; runtime governance latency overhead for the matched-configuration 9B and 20B comparisons is reported separately in Figure 10. System throughput is each system's peak over the evaluated micro-batches and includes the effect of enabled policies. The 35B and 120B results follow the projection methodology in §6.1. No workload-level padding correction is applied.

6.4 Microbenchmarks

6.4.1 Cost of Compile-Time Governance. Table 3 separates matched-configuration per-step throughput from the peak system throughput enabled by compile-time governance. At `micro-bs=1`, Qwen3.5-9B and GPT-OSS-20B retain 0.91–0.93× the baseline's single-user throughput under identical mappings; the corresponding governance overhead is 25–40 μs, or 7.6–9.5%, of TPOT in Figure 10. We use TPOT for governance-overhead claims because it directly measures the latency added to each decode step. The Qwen3.5-35B-A3B and GPT-OSS-120B rows include configuration changes and are not governance-only A/Bs.

The system columns answer the complementary policy-level question by reporting the best aggregate throughput for each system. Across these operating points, the expert-parallel and dynamically pipelined execution enabled by compile-time governance improves peak system throughput by 6.3–55.8×.

6.4.2 Ablation Study. Where Table 3 quantifies the cost of the governance substrate, Table 4 decomposes performance gains at the policy level. Dynamism-aware schedule synthesis and safe incremental materialization provide the substrate on which these policies execute. Decode adds expert parallelism, pipeline scheduling, and continuous batching. The current prefill implementation uses the same single-block FFN mapping as its static baseline, so its columns report only pipeline scheduling and request batching.

For decode, pipeline parallelism is the dominant factor (5.5–30.1×), followed by continuous batching (1.6–1.7×) and expert parallelism (1.2–1.4×). Without continuous batching, completed requests are padded until all requests in the same micro-batch finish. Continuous batching's factor at `micro-bs=4` is a conservative lower bound and grows with micro-batch size. For prefill on the three MoE models, pipeline scheduling at `micro-bs=1` contributes 4.0–6.9×.

Decode (`micro-bs=4`)

Model	**Expert**	**+ Pipe.**	**+ Cont.**	**Total**
GPT-OSS-20B	1.2×	20.0×	1.6×	**40.8**×
Qwen3.5-35B	1.2×	5.5×	1.7×	**10.7**×
GPT-OSS-120B	1.4×	30.1×	1.7×	**71.3**×

Prefill (seq = 4096)

Model	**Pipe. (`mbs`=1)**	**+ Req. (`mbs`=4)**	**Total (`mbs` : 1 → 4)**
GPT-OSS-20B	4.0×	1.6×	**6.6**×
Qwen3.5-35B	5.7×	1.1×	**6.4**×
GPT-OSS-120B	6.9×	1.6×	**10.8**×

Table 4. Policy-level decomposition of serving performance. In the table headers, `mbs` denotes micro-batch size. Each non-total column reports the multiplier over the preceding configuration after adding the named policy; the total is their product using unrounded values. Policy factors are derived from device-measured component cycles. Decode additionally uses the measured ShareGPT padding ratios for continuous batching. The prefill block reports the three MoE models: its first factor compares pipelined MeshRT at `micro-bs=1` with the static `bs=1` baseline, and the second factor moves MeshRT from `micro-bs=1` to `micro-bs=4`. The total therefore differs from Figure 9, which compares each system's peak over all evaluated micro-batches.

Increasing MeshRT from `micro-bs=1` to `micro-bs=4` adds 1.1–1.6×, for cumulative gains of 6.4–10.8× over the static `bs=1` baseline. These controlled cumulative gains are distinct from Figure 9's 4.0–6.6× 4K peak-to-peak range.

7 Related Work

Deterministic dataflow accelerators. The Groq TSP [1, 2], Graphcore IPU [23], and SambaNova RDU [52] all pre-compile execution schedules with no runtime scheduling overhead. The shared limitation is that any runtime uncertainty – such as variable batch size, MoE routing, request arrival / departure, forces execution off the fast path (host round-trip on IPU, unsupported on Groq). MeshRT occupies a different point: it pre-compiles per-core schedules that *govern* these dynamic events on-chip, preserving zero-runtime-scheduling while supporting continuous batching and expert routing. T10 [33] compiles individual DL operators onto IPU-like inter-core processors via the rTensor abstraction; MeshRT extends to full-model per-core schedules that handle cross-operator runtime dynamism.

MoEntwine [61] optimizes large-scale expert parallelism on wafer-scale chips, including expert placement, all-to-all congestion, and replica migration for load balancing. In contrast, MeshRT takes spatial mappings as input and provides a general on-chip control-plane mechanism for runtime dynamism, including MoE routing; the two approaches are therefore potentially composable. Tesseract [4] is a programmable processing-in-memory accelerator that sends

non-blocking remote function calls through message queues to the cores that own graph data, placing control requests alongside distributed data processing. It targets latency hiding and atomic updates in PIM graph processing rather than safe reconfiguration of a dynamically evolving compiled schedule on a GALS wafer. In contrast, MeshRT compiles runtime-dynamism handling into per-core schedules and safely reconfigures incrementally materialized segments to fit each core's bounded routing and buffering resources.

GPU persistent execution. Mirage [65] compiles tensor programs into mega-kernels via multi-level superoptimization, and CUDA Graphs [37] pre-record kernel sequences for replay—both reduce launch overhead on GPUs. Mirage is structurally the most comparable to MeshRT's always-on core execution. The key difference: GPU SMs support JIT dispatch and dynamic rebalancing via general-purpose control flow; WSE-3 PEs cannot, so MeshRT must resolve all scheduling at compile time.

LLM serving systems. Orca [66] introduces continuous batching; vLLM [28] adds paged KV cache management; SGLang [71] enables KV cache reuse via RadixAttention; TensorRT-LLM [40] provides production GPU kernels with in-flight batching. All assume a centralized host scheduler and shared GPU memory. MeshRT achieves continuous batching on mesh hardware where neither exists, using embedded control metainfo in compile-time schedules. DistServe [72] and Splitwise [51] disaggregate prefill and decode onto separate GPU pools; Sarathi-Serve [3] co-locates them with chunked prefill; Mooncake [53] disaggregates KV cache storage. LoongServe [64] dynamically adjusts sequence parallelism for long contexts.

DL compilers and parallelism frameworks. Ansor [69], Roller [73], and Triton [62] generate high-performance tensor programs for single-device shared-memory GPUs. Megatron-LM [36], Alpa [70], FlexFlow [24], and DeepSpeed-Inference [7] partition computation across discrete GPUs with GBs of HBM each, coordinated by a centralized scheduler over NVLink / InfiniBand. MeshRT targets a fundamentally different substrate: ~900K PEs with 48 KB each on a 2D mesh NoC, a three-order-of-magnitude difference in device count and per-device memory that demands a different approach to compilation, parallelism, and communication.

Wafer-scale and mesh architectures. The Cerebras WSE series [29, 31] provides the hardware substrate. WaferLLM [22] is the first published WSE inference system, using PE-level if-else branching among pre-compiled paths. MeshRT replaces branching with compile-time schedules that express continuous batching, expert-parallel MoE routing, and pipeline parallelism, capabilities beyond WaferLLM's branching model.

8 Discussion and Limitations

PE fault tolerance. MeshRT relies on the vendor-provided manufacturing-defect masking and remapping that uses redundant cores and routing to present a functional mesh [29]. Beyond that hardware layer, MeshRT does not detect or recover from PE failures that occur at runtime.

Multi-tenancy. MeshRT assumes exclusive access to the PE mesh for a single model. Serving multiple models concurrently on the same wafer: partitioning PE regions across models and potentially migrating them introduces resource management challenges not addressed by the current design.

Dynamic model loading. The current design assumes model weights are loaded onto PEs before inference begins and remain fixed. Hot-swapping model weights (*e.g.* for A/B testing or model updates) would require additional mechanisms for coordinated weight replacement.

Closed operator space assumption. MeshRT's completeness argument depends on the assumption that transformer inference decomposes into a finite, known set of mesh kernel primitives. Novel operator types (*e.g.* state-space models, linear attention variants) may require new kernel implementations and potentially new dynamism-handling mechanisms.

Globally coupled decisions and dependency stalls. Dynamism aware synthesis represents both local and globally coupled runtime decisions in the generated per-core schedules. For each mapped kernel and core, ReceivesDynamism and ProducesDynamism determine whether the compiler inserts a metainfo resolver before the kernel's data-processing sub-schedule or a metainfo producer after it. A global policy such as hot-expert load balancing is encoded as a chain of local load-statistic producers, a collective decision kernel, and downstream consumers; the same synthesis pass generates the role-specific schedules for all participants. This does not eliminate the collective's intrinsic aggregation cost. Moreover, a consumer's schedule must receive and install the required metainfo before dependent data movement begins; a delayed decision can therefore stall that fragment, and the compiled schedule cannot dynamically reorder unrelated work unless the overlap was encoded at compile time.

9 Conclusion

Wafer-scale accelerators offer compelling latency advantages for LLM inference, but existing software stacks sacrifice utilization by avoiding runtime dynamism. MeshRT resolves this through compile-time governance: for each compiled model, the compiler uses its finite, statically known set of mapped kernels to determine where dynamism arises and weave handling logic directly into per-core schedules, while safe incremental materialization ensures these schedules execute within tight hardware resource budgets. The result is a fully decentralized on-chip control path that preserves wafer-scale latency benefits while supporting continuous

batching, MoE routing, and micro-batch pipelining. On Cerebras CS-3 systems, MeshRT improves throughput by up to 56× over WaferLLM while adding 25–40 μs (7.6–9.5%) per decode token in matched configurations. Against measured single-instance SGLang results on H200, MeshRT delivers 4.5–7.0× lower decode latency; with ideal linear scaling to an area-normalized H200 cluster, it achieves up to 12.1× better decode energy efficiency.

References

[1] Dennis Abts, Garrin Kimmell, Andrew Ling, John Kim, Matt Boyd, Andrew Bitar, Sahil Parmar, Ibrahim Ahmed, Roberto DiCecco, David Han, John Thompson, Michael Bye, Jennifer Hwang, Jeremy Fowers, Peter Lillian, Ashwin Murthy, Elyas Mehtabuddin, Chetan Tekur, Thomas Sohmers, Kris Kang, Stephen Maresh, and Jonathan Ross. 2022. A software-defined tensor streaming multiprocessor for large-scale machine learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 567–580. doi:10.1145/3470496.3527405

[2] Dennis Abts, Jonathan Ross, Jonathan Sparling, Mark Wong-VanHaren, Max Baker, Tom Hawkins, Andrew Bell, John Thompson, Temesghen Kahsai, Garrin Kimmell, Jennifer Hwang, Rebekah Leslie-Hurd, Michael Bye, E. R. Creswick, Matthew Boyd, Mahitha Venigalla, Evan Laforge, Jon Purdy, Purushotham Kamath, Dinesh Maheshwari, Michael Beidler, Geert Rosseel, Omar Ahmad, Gleb Gagarin, Richard Czekalski, Ashay Rane, Sahil Parmar, Jeff Werner, Jim Sproch, Adrián Macías, and Brian Kurtz. 2020. Think Fast: A Tensor Streaming Processor (TSP) for Accelerating Deep Learning Workloads. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*. 145–158. doi:10.1109/ISCA45697.2020.00023

[3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 117–134.

[4] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. 2015. A scalable processing-in-memory accelerator for parallel graph processing. In *Proceedings of the 42nd annual international symposium on computer architecture*. 105–117.

[5] Joshua Ainslie, Tao Lei, Michiel de Jong, Santiago Ontañón, Siddhartha Brahma, Yury Zemlyanskiy, David C Uthus, Mandy Guo, James Lee-Thorp, Yi Tay, et al. 2023. Colt5: Faster long-range transformers with conditional computation. In *Proceedings of the 2023 conference on empirical methods in natural language processing*. 5085–5100.

[6] Amazon Web Services. 2025. Optimizing AI Responsiveness: Latency-Optimized Inference. https://aws.amazon.com/blogs/machine-learning/optimizing-ai-responsiveness-a-practical-guide-to-amazon-bedrock-latency-optimized-inference/

[7] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. 2022. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15. doi:10.1109/SC41404.2022.00051

[8] anon8231489123. 2023. ShareGPT_Vicuna_unfiltered: Cleaned and Filtered ShareGPT Conversations for Vicuna Training. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered.

[9] Anthropic. 2026. Introducing Claude Opus 5. https://www.anthropic.com/news/claude-opus-5.

[10] Cerebras. 2019. Wafer-Scale Deep Learning. In *2019 IEEE Hot Chips 31 Symposium (HCS)*. 1–31. doi:10.1109/HOTCHIPS.2019.8875628

[11] Cerebras Systems. 2024. Wafer-Scale Engine 3: The Largest Chip Ever Built. https://8968533.fs1.hubspotusercontent-na2.net/hubfs/8968533/Datasheets/WSE-3%20Datasheet.pdf. Product datasheet.

[12] K Mani Chandy and Leslie Lamport. 1985. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems (TOCS)* 3, 1 (1985), 63–75.

[13] Yu Chen, Runkai Chen, Sheng Yi, Xinda Zhao, Xiaohong Li, Jianjin Zhang, Jun Sun, Chuanrui Hu, Yunyun Han, Lidong Bing, et al. 2026. MSA: Memory Sparse Attention for Efficient End-to-End Memory Model Scaling to 100M Tokens. *arXiv preprint arXiv:2603.23516* (2026).

[14] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509* (2019).

[15] Claude AI. 2026. Claude Code Overview. https://code.claude.com/docs/en/overview

[16] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, et al. 2024. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1280–1297.

[17] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[18] Google Gemini. 2026. Gemini Cli. https://geminicli.com/

[19] Google Gemini. 2026. Try Deep Research and our new experimental model in Gemini, your AI assistant. https://blog.google/products-and-platforms/products/gemini/google-gemini-deep-research/

[20] Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, et al. 2025. Openthoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178* (2025).

[21] Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. 2024. DS-agent: automated data science by empowering large language models with case-based reasoning. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Article 668, 36 pages.

[22] Congjie He, Yeqi Huang, Pei Mu, Ziming Miao, Jilong Xue, Lingxiao Ma, Fan Yang, and Luo Mai. 2025. WaferLLM: Large Language Model Inference at Wafer Scale. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 257–273.

[23] Zhe Jia, Blake Tillman, Marco Maggioni, and Daniele Paolo Scarpazza. 2019. Dissecting the Graphcore IPU Architecture via Microbenchmarking. *arXiv preprint arXiv:1912.03413* (2019).

[24] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond Data and Model Parallelism for Deep Neural Networks. In *Proceedings of the 2nd Conference on Machine Learning and Systems (MLSys)*. Palo Alto, CA.

[25] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).

[26] Bentian Jiang, Jingsong Chen, Jinwei Liu, Lixin Liu, Fangzhou Wang, Xiaopeng Zhang, and Evangeline FY Young. 2020. CU. POKer: Placing DNNs on wafer-scale AI accelerator with optimal kernel sizing. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–9.

[27] Sheng-Chun Kao and Tushar Krishna. 2020. GAMMA: Automating the HW Mapping of DNN Models on Accelerators via Genetic Algorithm. In *2020 IEEE/ACM International Conference On Computer Aided Design*

(ICCAD). 1–9.

[28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. ACM. doi:10.1145/3600006.3613165

[29] Gary Lauterbach. 2021. The Path to Successful Wafer-Scale Integration: The Cerebras Story. *IEEE Micro* 41, 6 (2021), 52–57. doi:10.1109/MM.2021.3112025

[30] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) *(ICML'23)*. JMLR.org, Article 795, 13 pages.

[31] Sean Lie. 2023. Cerebras Architecture Deep Dive: First Look Inside the Hardware/Software Co-Design for Deep Learning. *IEEE Micro* 43, 3 (May 2023), 18–30. doi:10.1109/MM.2023.3256384

[32] Sean Lie. 2024. Wafer-Scale AI: GPU Impossible Performance. In *2024 IEEE Hot Chips 36 Symposium (HCS)*. doi:10.1109/HCS61935.2024.10664673

[33] Yiqi Liu, Yuqi Xue, Yu Cheng, Lingxiao Ma, Ziming Miao, Jilong Xue, and Jian Huang. 2024. Scaling Deep Learning Computation over the Inter-Core Connected Intelligence Processor with T10. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*. ACM, 505–521. doi:10.1145/3694715.3695955

[34] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 932–949.

[35] Microsoft. 2026. Microsoft Copilot. https://copilot.microsoft.com/

[36] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. 1–14. doi:10.1145/3458817.3476209

[37] NVIDIA. 2018. CUDA Graphs. https://developer.nvidia.com/blog/cuda-10-features-revealed/. Introduced in CUDA 10.

[38] NVIDIA. 2022. NVIDIA Hopper Architecture In-Depth. https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/.

[39] NVIDIA. 2024. NVIDIA H200 Tensor Core GPU. https://www.nvidia.com/en-us/data-center/h200/. Product specifications.

[40] NVIDIA. 2024. TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM.

[41] Hyungjun Oh, Kihong Kim, Jaemin Kim, Sungkyun Kim, Junyeol Lee, Du-seong Chang, and Jiwon Seo. 2024. Exegpt: Constraint-aware resource scheduling for llm inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 369–384.

[42] OpenAI. 2025. GPT-OSS-120B. https://huggingface.co/openai/gpt-oss-120b. GPT-OSS-120B.

[43] OpenAI. 2025. GPT-OSS-20B. https://huggingface.co/openai/gpt-oss-20b. GPT-OSS-20B.

[44] OpenAI. 2026. Codex. https://openai.com/codex/

[45] OpenAI. 2026. Deep research in ChatGPT. https://chatgpt.com/features/deep-research/

[46] OpenAI. 2026. Introducing GPT-5.3-Codex-Spark. https://openai.com/index/introducing-gpt-5-3-codex-spark/.

[47] OpenAI. 2026. Previewing Ultrafast Mode: GPT-5.6 Sol at up to 14X the Speed. https://openai.com/index/previewing-ultrafast/.

[48] OpenClaw. 2026. OpenClaw. https://openclaw.ai/

[49] Mihrimah Ozkan, Lily Pompa, Md Shaihan Bin Iqbal, Yiu Chan, Daniel Morales, Zixun Chen, Handing Wang, Lusha Gao, and Sandra Hernandez Gonzalez. 2025. Performance, efficiency, and cost analysis of wafer-scale AI accelerators vs. single-chip GPUs. *Device* 3, 10 (2025), 100834. doi:10.1016/j.device.2025.100834

[50] Bowen Pang, Kai Li, and Feifan Wang. 2025. Optimizing LLM inference throughput via memory-aware and SLA-constrained dynamic batching. *arXiv preprint arXiv:2503.05248* (2025).

[51] Pratyush Patel, Esha Choukse, Chaojie Zhang, Í nigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA59077.2024.00019

[52] Raghu Prabhakar et al. 2024. SambaNova SN40L: Scaling the AI Memory Wall with Dataflow and Composition of Experts. *arXiv preprint arXiv:2405.07518* (2024).

[53] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 155–170.

[54] Qwen. 2026. Qwen3.5-35B-A3B. https://huggingface.co/Qwen/Qwen3.5-35B-A3B. Qwen3.5-35B-A3B.

[55] Qwen. 2026. Qwen3.5-9B. https://huggingface.co/Qwen/Qwen3.5-9B. Qwen3.5-9B.

[56] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*. PMLR, 18332–18346.

[57] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258* (2024).

[58] Agnia Sergeyuk, Yaroslav Golubev, Timofey Bryksin, and Iftekhar Ahmed. 2025. Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward. *Information and Software Technology* 178 (2025), 107610.

[59] SGLang. 2026. SGLang v0.5.9. https://github.com/sgl-project/sglang/releases/tag/v0.5.9. SGLang 0.5.9.

[60] Qidong Su, Wei Zhao, Xin Li, Muralidhar Andoorveedu, Chenhao Jiang, Zhanda Zhu, Kevin Song, Christina Giannoula, and Gennady Pekhimenko. 2025. Seesaw: High-throughput llm inference via model re-sharding. *Proceedings of Machine Learning and Systems* 7 (2025).

[61] Xinru Tang, Jingxiang Hou, Dingcheng Jiang, Taiquan Wei, Jiaxin Liu, Jinyi Deng, Huizheng Wang, Qize Yang, Haoran Shang, Chao Li, et al. 2026. MoEntwine: Unleashing the potential of wafer-scale chips for large-scale expert parallel inference. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–15. doi:10.1109/HPCA68181.2026.11408594

[62] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*. ACM, 10–19. doi:10.1145/3315508.3329973

[63] Sally Ward-Foxton. 2024. Cerebras' Third-Gen Wafer-Scale Chip Doubles Performance. https://www.eetimes.com/cerebras-third-gen-wafer-scale-chip-doubles-performance/. EE Times.

[64] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*

(SOSP). ACM. doi:10.1145/3694715.3695948

[65] Mengdi Wu, Xinhao Cheng, Shengyu Liu, Chunan Shi, Jianan Ji, Kit Ao, Praveen Velliengiri, Xupeng Miao, Oded Padon, and Zhihao Jia. 2025. Mirage: A Multi-Level Superoptimizer for Tensor Programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 221–238.

[66] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538.

[67] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, et al. 2025. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 23078–23097.

[68] Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. 2025. Spargeattention: Accurate and training-free sparse attention accelerating any model inference. *arXiv preprint arXiv:2502.18137* (2025).

[69] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 863–879.

[70] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 559–578.

[71] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*. 62557–62583.

[72] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210.

[73] Hongyu Zhu, Ruofan Wu, Yijia Diao, Shanbin Ke, Haoyu Li, Chen Zhang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Wei Cui, Fan Yang, Mao Yang, Lidong Zhou, Asaf Cidon, and Gennady Pekhimenko. 2022. ROLLER: Fast and Efficient Tensor Compilation for Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 233–248.

[74] Jingchen Zhu, Chenhao Xue, Yiqi Chen, Zhao Wang, Chen Zhang, Yu Shen, Yifan Chen, Zekang Cheng, Yu Jiang, Tianqi Wang, et al. 2025. Theseus: Exploring efficient wafer-scale chip design for large language models. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).